RAG Assignment: Baseline vs. Re-Ranking

Gargi Saini · Language Models and Agents (LMA), Assignment 1 · Code: github.com/gargisaini/rag-baseline-vs-reranking

Setup

For the document I picked **NIST SP 800-63B-4**, the US government's guideline on authentication and passwords (129 pages, July 2025). I chose it because it is written as a long list of "SHALL", "SHALL NOT" and "MAY ... but" rules, and several rules show up more than once: in the formal requirements, and again, more loosely, in the usability section and in Appendix A. That felt like a fair place to look for retrieval mistakes.

I dropped the cover and table of contents (TOC lines are section titles that would match queries without containing any actual rules), removed the header that repeats on every page, and cut the text into chunks of 200 tokens with 40 tokens of overlap, which gave 337 chunks. I measured chunk size in the bi-encoder's own tokens on purpose. My first attempt used 180-word windows, and some of those turned into 500+ tokens, which the bi-encoder (256-token limit) would have silently cut off while the cross-encoder still read them. That would have made the baseline look worse than it really is.

Models: `all-MiniLM-L6-v2` as the bi-encoder with an exact FAISS inner-product index, `cross-encoder/ms-marco-MiniLM-L-6-v2` as the re-ranker, and `gemini-3.5-flash` (temperature 0) to write the answers. Both pipelines use the same prompt function: three numbered passages, cite them like [1], and say "I don't know" if the answer isn't there. The only thing that changes between Answer 1 and Answer 2 is which three chunks go in.

1. Architecture comparison

A **bi-encoder** reads the query and each chunk separately and turns each one into a single vector. Relevance is just the cosine between the two vectors. Since chunk vectors don't depend on the query, I can compute all of them once, store them in an index, and at question time I only have to embed the query and do a fast vector search. The catch is that the whole chunk gets squeezed into one vector before it ever sees the question, so the score mostly reflects overall topic overlap.

A **cross-encoder** takes the query and the chunk together as one input, so self-attention runs across both texts and every query word can look at every chunk word. It outputs one relevance score for that pair. This lets it pick up things a single vector blurs, such as whether the chunk actually answers the question, or whether a "SHALL NOT" applies to the thing being asked about. The cost is that nothing can be precomputed: every new query needs a full transformer pass for every chunk you want to score.

2. Qualitative & rank-shift analysis

I wrote 10 questions the way a normal user would phrase them, not in the document's wording, and marked the chunk(s) that really contain the answer as "gold". Here is where the best gold chunk sat before and after re-ranking (top 3 is what the LLM gets to see):

Type	Question	Gold chunk: baseline rank → re-ranked rank	Result
Keyword trap	Can a website force me to include special characters and numbers in my password?	chunk 40: #7 → #2	**rescued**
Keyword trap	Do users have to change their password every few months?	chunk 219: #5 → #1	**rescued**
Negation	Are password hints allowed?	chunk 40: #5 → #1	**rescued**
Exception	What is the minimum password length?	chunk 39: #5 → #1	**rescued**
Buried rule	Is pasting a password into the login field allowed?	chunk 44: #4 → #1	**rescued**
Constraint	Can a fingerprint alone be used to log in?	chunk 99: #2 → #2	already in top 3
Buried rule	How many wrong guesses can an attacker make before the account is locked?	chunk 94: #2 → #1	already in top 3
Negation	Are security questions allowed when choosing a password?	not in top 10 (bi-encoder rank #11)	recall miss
Exception	Can SMS be used for out-of-band authentication?	not in top 10 (bi-encoder rank #14)	recall miss
Buried rule	Can a verifier check only part of the password?	not in top 10 (bi-encoder rank #18)	recall miss

So the cross-encoder pulled the answer into the top 3 in 5 of the 10 questions, never pushed a gold chunk out of the top 3, and could do nothing for the 3 questions where the bi-encoder had already dropped the answer below rank 10. The cases that taught me the most:

Keyword trap: "Can a website force me to include special characters and numbers?"

The baseline's top 3 were chunks 269 and 264 from Appendix A ("Strength of Passwords") plus chunk 263, the end of the reference list running into that appendix. Appendix A *describes* how many websites force rules like "an uppercase letter and a number", so it is full of words from my question. The actual rule, "Verifiers and CSPs SHALL NOT impose other composition rules", sits in chunk 40, which the bi-encoder put at #7. The cross-encoder moved it to #2.

> **ANSWER 1 (BASELINE)**
> "Yes. Online services use 'composition rules' that require users to construct passwords using a mix of character types, such as at least one digit (number), uppercase letter, and symbol [1], [2]."
>
> **ANSWER 2 (RE-RANKED)**
> "...verifiers and Credential Service Providers (CSPs) **shall not** impose composition rules requiring mixtures of different character types [2]. However, passage [1] notes that in practice, some online services do require users to include uppercase letters, numbers, or symbols [1]."

This is the clearest failure in the whole set. Answer 1 isn't made up, since every sentence is supported by a passage, but it states the rule backwards, because the LLM only saw passages describing the bad practice. Answer 2 gets the rule right and even separates "what the guideline says" from "what websites do".

Exception: "What is the minimum password length?"

The bi-encoder's #1 was chunk 266 from the appendix, which says the right length "depends on the threat model being addressed". The chunk that has the numbers (chunk 39) was at #5. After re-ranking it is #1

with a clear lead (CE score 6.27 against 5.55 for the next one).

> **ANSWER 1 (BASELINE)**
> "...a specific numerical minimum password length is not provided; rather, the required minimum password length 'depends on the threat model being addressed' [1]."
>
> **ANSWER 2 (RE-RANKED)**
> "Single-factor authentication: passwords must be a minimum of 15 characters... Multi-factor authentication: passwords may be shorter, but must be a minimum of eight characters [1]."

Answer 1 completely misses the essential numbers, and Answer 2 also catches the exception (8 characters when the password is only one part of multi-factor).

Omitted clause: "Do users have to change their password every few months?"

This one surprised me. Both answers say "no", so at first glance the baseline looks fine. But the baseline's #2 chunk (218) happens to end in the middle of the sentence, at "Do not require that passwords be changed arbitrarily (e.g.,". The next chunk (219), which contains the rest, "...unless there is a user request or evidence of authenticator compromise", was at #5. The cross-encoder put 219 at #1.

> **ANSWER 1 (BASELINE)**
> "Based on the provided text, no. The guidelines state: 'Do not require that passwords be changed arbitrarily' [2]."
>
> **ANSWER 2 (RE-RANKED)**
> "No. Passwords should not be required to be changed arbitrarily or periodically unless there is a user request or evidence of an authenticator compromise [1][2]."

Answer 1 drops the exception, and in a security policy the exception is the important part: you *do* have to force a change after a compromise.

Negation: "Are password hints allowed?"

The answer, "SHALL NOT permit the subscriber to store a hint", is item 7 in the same list as the composition rule (chunk 40), and the bi-encoder had it at #5 behind general password-advice chunks. Answer 1 was "I don't know. The provided passages do not mention whether password hints are allowed." Answer 2, with chunk 40 at #1, said "No, verifiers and CSPs shall not permit subscribers to store a password hint that is accessible to an unauthenticated claimant [1]." Good to see that the "I don't know" instruction worked here instead of the model guessing.

Buried rule: "Is pasting a password allowed?"

Here both answers were "yes", so re-ranking didn't change the conclusion. The difference is in the evidence. The baseline answer leaned on a tip from the usability section, while the re-ranked answer cited the actual requirement in chunk 44 (moved #4 → #1): verifiers "SHOULD permit claimants to use the 'paste' function". I'd count it as a quieter improvement: same answer, better source.

Where re-ranking could not help

For the security-question, SMS and truncation questions, the gold chunk was at bi-encoder rank #11, #14 and #18, just outside the top 10, so the cross-encoder never saw it. The truncation and security-question answers were both "I don't know" under both pipelines. The SMS case was a bit worrying. Answer 2 confidently said "Yes, SMS can be used", because re-ranking brought in a chunk mentioning SMS, but neither pipeline saw the sentence saying PSTN/SMS use is *restricted*. So a better ordering of the wrong

candidates made the answer sound more sure, not more correct. My takeaway is the one from the lecture: re-ranking fixes ranking, not recall. Pulling 30 to 50 candidates instead of 10, or adding BM25 for hybrid search, would likely fix these three, since all of them were close to the cut-off.

3. Latency profiling

Measured on CPU after a warm-up run. Retrieval stages are the median of 10 questions × 5 repeats; LLM time is the median of the real Gemini calls.

Stage (median, ms)	Pipeline 1: Baseline	Pipeline 2: Re-ranking
Query embedding + top-10 FAISS search	13.3	13.3
Cross-encoder on 10 (query, chunk) pairs	–	285.2
Retrieval total	**13.3**	**298.4**
LLM generation (Gemini API)	4,922.1	4,482.4
End-to-end	**4,935.3**	**4,780.8**

Re-ranking makes retrieval about 22× slower (13 ms → 298 ms), which sounds like a lot until you see the LLM call takes 4 to 5 seconds. The extra 285 ms is about 6% of a full request. The LLM rows differ only because of network noise. Both pipelines send the same prompt size, and Pipeline 2 even came out faster end to end in this run. For a user, the re-ranker's cost is basically invisible next to generation.

4. Scaling question: why not run the cross-encoder on everything?

Because the cross-encoder has to do a full forward pass for every (query, chunk) pair, and none of that can be done ahead of time since every score depends on the query. The bi-encoder's expensive part, embedding the chunks, happens once at indexing time, so a query costs one small embedding plus a vector search. I measured both on my 337 chunks and scaled them linearly:

Per query	337 chunks (measured)	1 million chunks (linear estimate)
Bi-encoder: embed query + search every vector	19.3 ms	754 ms
Cross-encoder: score every chunk	10,767 ms	532.5 min (~9 hours)

Even on this small document, scoring everything with the cross-encoder takes almost 11 seconds per question, more than double the LLM call. At a million chunks it would be about nine hours *per question*, while the bi-encoder side would still be under a second, and much less with an approximate index like HNSW. That's why the two are used together: the bi-encoder is cheap enough to search everything and get a decent shortlist (recall), and the cross-encoder is accurate enough to put the right few at the top of that shortlist (precision). Running the cross-encoder only on the 10 candidates cost me 285 ms, about 38 times less than scoring the whole document.